

Custom Deep Learning Framework and Autoencoder Implementation

NAME	ID
Nour Emad Rabea	2101443
Abdulrahman Khaled	2101779
Mohamed Reda	2100459

Submitted To:

Dr. Hossam Hassan

Eng. Abdallah Mohammed

1. Introduction

While high-level deep learning libraries abstract away complex mathematical operations, building these algorithms from scratch is essential to truly master the mechanics of neural networks. This section presents Part 1 of the project: a custom, object-oriented framework developed entirely in Python using purely NumPy for efficient matrix operations.

The developed library seamlessly handles linear transformations for dense layers, dropout regularization, analytical derivatives for various activation functions, backpropagation logic, and Stochastic Gradient Descent (SGD) optimization with momentum. To validate its mathematical integrity and forward-backward propagation flow, the framework is rigorously tested using numerical gradient checking and successfully evaluated against the non-linear XOR problem.

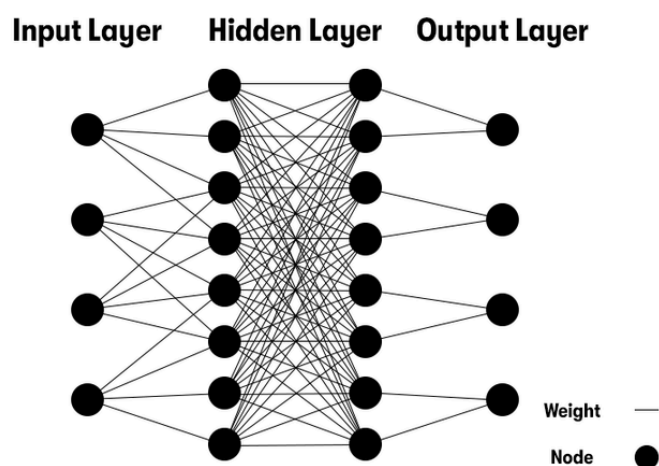


Figure 1: Neural Network Architecture — Input, Hidden, and Output Layers

2. Objectives

Demystifying Deep Learning Mechanics

To construct a comprehensive, object-oriented neural network framework from scratch utilizing exclusively NumPy, ensuring a profound command over low-level linear transformations, activation function derivatives, and analytical error gradients during backpropagation.

Mathematical Verification

To rigorously validate the internal logic and backward flow of the custom framework through numerical gradient checking, ensuring flawless analytical derivative calculations.

Validating Non-Linear Learning Capabilities

To evaluate the framework's training stability and optimization efficiency by successfully resolving the non-linearly separable XOR problem using the implemented Stochastic Gradient Descent (SGD) with momentum.

3. Framework Architecture

The framework employs a modular, object-oriented architecture where distinct components—such as layers, activations, and optimizers—are orchestrated by a central Network conductor to seamlessly manage forward passes and backpropagation.

3.1 Layer Abstractions

Base Layer

The Layer class serves as an abstract blueprint for all network components. It enforces a strict contract where every subclass must systematically implement a `forward()` pass for data computation and a `backward()` pass for gradient calculation via the chain rule.

Dense Layer

This class implements a fully connected layer responsible for the linear transformation shown below. To ensure stable gradient flow and mitigate the vanishing or exploding gradient problems, the layer incorporates robust weight initialization strategies, specifically He initialization and Xavier initialization.

$$Y = X \cdot W + b$$

$$\text{He Initialization: } W \sim N(0, \sqrt{2 / N_{in}})$$

$$\text{Xavier Initialization: } W \sim N(0, \sqrt{1 / N_{in}})$$

During the `backward()` pass, the layer analytically computes the gradients with respect to the weights ($\partial L / \partial W$) and biases ($\partial L / \partial b$), while caching the inputs during the forward pass to optimize memory and computation.

Dropout Layer

Acting as a crucial regularizer, this layer prevents overfitting by randomly disabling a fraction of neurons during training. It utilizes a binomial distribution mask via `np.random.binomial` to drop units based on a specified rate. The implementation efficiently utilizes the "Inverted Dropout" technique; by scaling the surviving neurons during training, it ensures the expected sum remains constant, eliminating the need to scale outputs during the testing or evaluation phases.

$$\text{Output}_{train} = (\text{mask} / (1 - \text{rate})) \cdot X$$

3.2 Activation Functions

Activation functions introduce complex, non-linear mappings into the network. In this framework, they are implemented as subclasses of the base Layer, explicitly defining their analytical derivatives in the backward() pass to seamlessly integrate with the chain rule during backpropagation.

ReLU (Rectified Linear Unit)

Implements the following forward pass, outputting the input directly if positive and zero otherwise. During the backward pass, it acts as a gradient switch, propagating gradients only where the original input was strictly greater than zero using a boolean mask.

$$f(x) = \max(0, x)$$

$$f'(x) = 1 \text{ if } x > 0, \text{ else } 0$$

Sigmoid

Maps the input to a probability range of (0, 1). To optimize computational efficiency, the forward pass caches the output $f(x)$. The backward pass then reuses this cached value to compute the local gradient analytically.

$$f(x) = 1 / (1 + e^{-x})$$

$$f'(x) = f(x) \cdot (1 - f(x))$$

Tanh (Hyperbolic Tangent)

Maps the input to a range of (-1, 1). Similar to the Sigmoid implementation, the forward output is cached, and the backward pass efficiently computes the derivative using the following property.

$$f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$$

$$f'(x) = 1 - f(x)^2$$

Softmax

Typically utilized in the output layer for multi-class classification to yield a valid probability distribution. A critical design choice in this implementation is the optimization for numerical stability; it subtracts the maximum input value along the feature axis before exponentiation to prevent overflow errors. For architectural simplicity, the explicit local derivative calculation is bypassed in the backward() pass (return grad_output), as the framework couples the Softmax gradient computation directly within the Categorical Cross-Entropy (CCE) loss function.

$$f(x_i) = e^{(x_i - \max(x))} / \sum e^{(x_j - \max(x))}$$

3.3 Loss Functions

The library supports various objective functions to quantify the network's prediction error, tailored for both regression and classification tasks. Every loss class implements a `forward()` method to compute the scalar error and a `backward()` method to initiate the backpropagation process by calculating the initial gradient ($\partial L / \partial Y_{pred}$).

Mean Squared Error (MSE)

Utilized primarily for regression and image reconstruction tasks (e.g., the Autoencoder). It calculates the average of the squared differences between predictions and true values. The analytical gradient is explicitly computed as follows.

$$MSE = (1/N) \cdot \Sigma (Y_{pred} - Y_{true})^2$$

$$\partial MSE / \partial Y_{pred} = (2/N) \cdot (Y_{pred} - Y_{true})$$

Binary Cross-Entropy (BCE)

Designed for binary classification problems. To prevent severe numerical instability—specifically, computing the logarithm of zero which results in NaN or infinity—the predictions are mathematically clipped before loss calculation.

$$BCE = -(1/N) \cdot \Sigma [y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

$$\hat{y} \in [1e-15, 1 - 1e-15]$$

Categorical Cross-Entropy (CCE)

Implemented for multi-class classification. Similar to BCE, it employs numerical clipping for stability. A significant architectural optimization in this framework is its pairing with the Softmax activation; the `backward()` pass directly computes the combined, simplified gradient, bypassing redundant and computationally expensive derivative calculations.

$$CCE = -(1/N) \cdot \Sigma \Sigma Y_{true} \cdot \log(Y_{pred})$$

$$\partial CCE / \partial Y_{pred} = (1/N) \cdot (Y_{pred} - Y_{true})$$

3.4 Optimizer: SGD with Momentum

To update the network's learnable parameters and minimize the loss function, the framework implements an advanced Stochastic Gradient Descent (SGD) optimizer augmented with Momentum. Unlike standard SGD, which updates parameters based solely on the immediate gradient, this implementation tracks the exponentially weighted averages of past gradients (velocities) for both weights and biases. This mechanism significantly accelerates convergence and dampens oscillations across epochs.

During the optimization step, the `update()` method iterates through the network layers, utilizing introspection (`hasattr`) to isolate layers with learnable parameters (e.g., Dense layers) from parameter-less layers (e.g., Activations, Dropout). For each applicable layer, it updates the velocity and parameter θ using the following rules:

$$v = \gamma \cdot v + \eta \cdot \nabla \theta$$

$$\theta = \theta - v$$

Where γ is the momentum coefficient, η is the learning rate, and $\nabla \theta$ is the gradient computed during backpropagation.

3.5 Network Class

The Network class acts as the central conductor of the framework, storing the network architecture (layers), the objective function, and the optimizer. It abstracts the training complexity into a unified interface, primarily through the `train_step` method.

During a single training iteration, the `train_step` sequentially orchestrates four critical operations:

1. **Forward Pass:** It iteratively passes the input batch through the stacked layers to generate the final predictions.
2. **Loss Calculation:** It evaluates these predictions against the true labels using the specified loss function to compute the scalar error.
3. **Backpropagation:** It retrieves the initial gradient from the loss function and propagates it backward through the network architecture in reverse order, systematically calculating local gradients for each layer.
4. **Parameter Update:** It commands the SGD optimizer to traverse the layers and adjust the learnable weights and biases based on the newly computed gradients.

4. Experiments and Validation

The architecture was put through a series of tests to validate its logic and predictive power.

4.1 Gradient Checking

To rigorously verify the mathematical correctness of the framework's backpropagation logic, a numerical gradient checking procedure was conducted. This test ensures that the analytical derivatives computed via the chain rule during the backward() passes accurately match the numerical approximation of the gradients.

The numerical gradient was calculated using the central difference approximation:

$$\partial L / \partial W \approx [L(W + \epsilon) - L(W - \epsilon)] / (2\epsilon)$$

A test scenario was configured utilizing a single Dense(2, 1) layer and the MSE loss function. A specific weight parameter ($W_{0,0}$) was isolated, and the perturbation value (ϵ) was set to $1e^{-5}$.

```
Analytical Gradient (Backprop): -0.37291436
Numerical Gradient (Formula):   -0.37291436
Difference:                     4.95221086e-11
```

The extremely marginal difference of approximately 4.95×10^{-11} validates that the analytical gradients are correct, confirming that the framework's internal derivative calculations and backward flow are flawlessly implemented.

4.2 XOR Problem

The XOR (Exclusive OR) logical operation is a classic benchmark in machine learning. Because its data points are not linearly separable, a simple single-layer perceptron cannot solve it. Successfully classifying XOR inputs proves that a neural network can map complex, non-linear relationships through its hidden layers and non-linear activation functions.

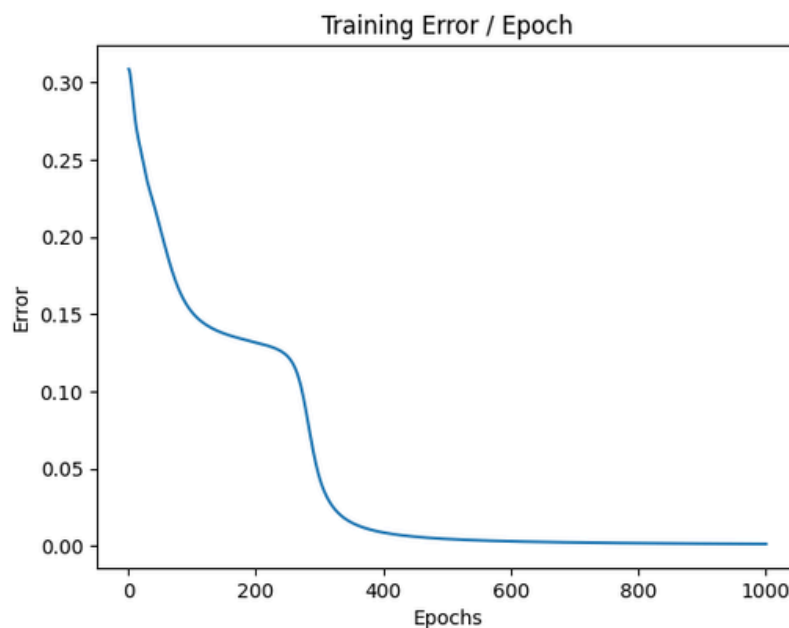
A non-linear XOR dataset was used to rigorously test the custom framework's learning capabilities. The network architecture consisted of:

- Hidden Layer: Dense(2, 4) with Tanh activation.
- Output Layer: Dense(4, 1) with Sigmoid activation.

The model was trained for 1,000 epochs using the Mean Squared Error (MSE) loss function and the SGD optimizer with a learning rate of 0.1 and momentum of 0.9. As demonstrated by the training error curve, the loss smoothly minimized over time, indicating stable convergence.

```
--- Final Results ---  
Input: [0 0] | True: 0 | Predicted: 0.0201  
Input: [0 1] | True: 1 | Predicted: 0.9617  
Input: [1 0] | True: 1 | Predicted: 0.9685  
Input: [1 1] | True: 0 | Predicted: 0.0410
```

The final predictions were highly accurate relative to the truth table, successfully validating the forward and backward propagation logic.



5. Conclusion

The development of this custom neural network framework successfully demystified the underlying mechanics of deep learning. By constructing the architecture entirely from scratch utilizing NumPy, the mathematical rigor of forward propagation, analytical derivatives, and optimization algorithms was validated without relying on high-level commercial abstractions.

The framework demonstrated robust training stability and algorithmic accuracy by successfully solving the non-linearly separable XOR problem. Furthermore, rigorous numerical gradient checking confirmed that the custom backpropagation logic is mathematically sound and flawlessly implemented.

Ultimately, this foundational phase bridges the gap between theoretical calculus and practical algorithmic implementation. The modular, object-oriented design established here provides a highly reliable architecture, perfectly positioning the framework for advanced, large-scale machine learning tasks and complex feature extraction in future applications.

6. Repository

The full source code for this project is available on GitHub: [Mohamed-AbdelGhaffar/CI_Project \(main branch\)](#).